

MACHINE LEARNING • CONSULTING

Graph-Based Network Intelligence

Optimizing Delivery ETAs for Delhivery's logistics network

Om Sharma | Lakshya Chahar | Mayank Bagoria

A directed-graph approach to ETA prediction, bottleneck detection, and route-type strategy

Dataset: Delhivery feature-engineering data, September–October 2018

Contents

1	Introduction and Motivation	2
2	System Overview	2
2.1	The end-to-end flow	2
2.2	Why a graph	3
2.3	Module and artifact structure	3
3	Technology Stack	3
4	Data Pipeline	4
4.1	The data model	4
4.2	Collapsing scans to legs	4
4.3	Cleaning and feature engineering	4
4.4	What the cleaned data says	5
5	Graph Construction	5
5.1	Nodes and edges	5
5.2	Edge weights and stratification	5
5.3	Connectivity	6
6	Bottleneck and Corridor Audit	6
6.1	Centrality measures and what they mean	6
6.2	The composite bottleneck score	6
6.3	Top bottleneck hubs	6
6.4	Chronically delayed corridors	8
7	Graph-Enhanced ETA Prediction	8
7.1	Target and features	8
7.2	Keeping the comparison honest	8
7.3	Models and results	9
7.4	What drives the model	9
8	FTL versus Carting Decision Framework	10
8.1	The confounding problem	10
8.2	The cost model	10
8.3	The generalized cost and decision	10
9	A Worked Example: Gurgaon → Bangalore	11
10	The Interactive Dashboard	12
11	Design Decisions	13
12	Business Impact and Recommendations	14
13	Limitations and Future Work	14
14	Reproducibility and Conclusion	15
A	Appendix: Worst Chronic Corridors	15
B	Appendix: Environment and Setup	15

AT A GLANCE. The OSRM routing engine under-predicts leg time by a median **2.0×**; delay is concentrated, with the **top five hubs holding 30.8%** of all network delay. A graph-enhanced ETA model is on-target (within 15%) **58% of the time versus 4%** for OSRM, and upgrading the top three hubs recovers an estimated **~Rs 1.5 crore/year** of revenue-at-risk.

1 Introduction and Motivation

Delhivery is India's largest fully-integrated logistics provider, operating a vast network of facilities, inter-city routes, and last-mile delivery across every major state. At the core of its operations is a hub-and-spoke model: a shipment travels from a source facility through one or more intermediate hubs before reaching its destination, and each hop is a single *leg* of a multi-leg journey.

To promise delivery times, Delhivery uses OSRM, a standard routing engine that assumes clean traffic and shortest paths. Real-world logistics is far messier. Congestion, facility dwell time, seasonal volume spikes and route-type constraints all cause actual delivery times to deviate substantially from the prediction. When the ETA is wrong, SLAs are missed and customers are unhappy; downstream capacity planning breaks across the network; and there is no systematic way to see which hubs and corridors are the biggest contributors to delay.

The strategic question this report answers is whether a *graph-based* model — one that treats the logistics network as a connected graph of facilities and corridors rather than a collection of independent point-to-point estimates — can produce more accurate ETAs and identify which corridors and hubs systematically cause delays. The deliverable is not just a model: it is a decision system, ending in a strategy an operations leader can act on.

We work from a public Delhivery dataset of 144,867 scan events spanning 14,817 trips over roughly 25 days. After cleaning, this yields 26,258 leg-level corridor traversals across 1,657 facilities and 2,775 corridors — the raw material for the graph.

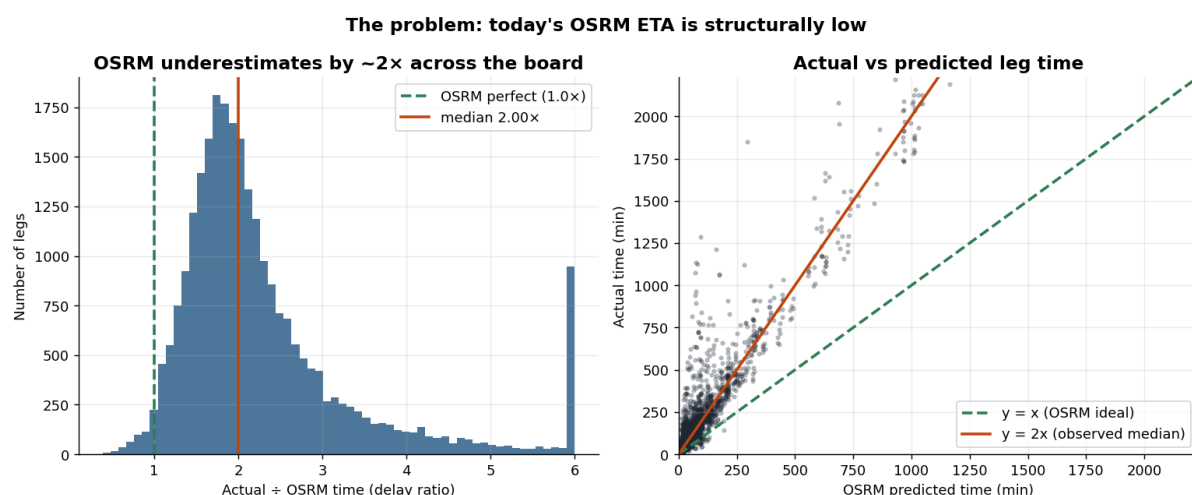


Figure 1. The OSRM estimate is systematically $\sim 2\times$ too fast. Left: the distribution of the actual÷OSRM delay ratio (median 2.0). Right: actual versus predicted leg time — the cloud sits on the $y = 2x$ line, not $y = x$.

Why this matters. The literal “> 20% over OSRM” SLA-breach flag fires on 94.6% of legs — it is near-universal and therefore not discriminating. The operational signal lives in the *severity* (how many times over) and the *volume* (how many shipments), so throughout we rank by total excess delay minutes rather than the binary flag. This single reframing is what turns a vague “OSRM is sometimes wrong” into a ranked, actionable list of fixes.

2 System Overview

2.1 The end-to-end flow

A run travels through seven ordered stages. Raw scan events are collapsed into clean leg-level corridor traversals; the legs build a directed weighted graph; the graph drives three parallel analyses (bottleneck audit, ETA model, route-type framework); and the findings are packaged as an operations memo and an interactive dashboard. The two shaded stages below are machine-learning models; everything else is deterministic data logic.

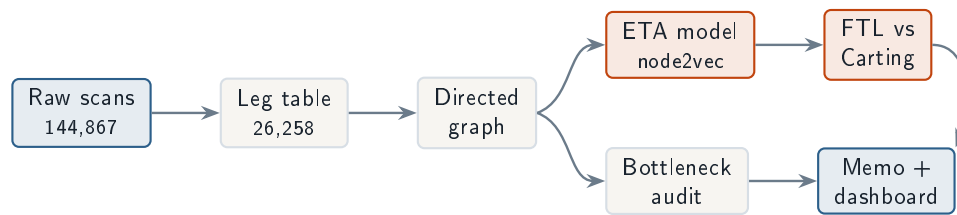


Figure 2. The pipeline. Shaded boxes are model stages (gradient-boosted trees with node2vec graph embeddings); the rest are deterministic transforms. Modules run in numeric order under a fixed seed.

2.2 Why a graph

Modelling the network as a graph is what lets “where” enter the analysis. A point-to-point estimate cannot know that a corridor inherits congestion from the hub that feeds it, or that one facility sits on a disproportionate share of the country’s least-time routes. Encoding facilities as nodes and corridors as weighted directed edges makes those structural facts first-class: betweenness identifies chokepoints, corridor history becomes a feature, and node embeddings let the ETA model generalise across structurally-similar facilities. The same structure also supports counterfactual reasoning about route type, because a facility’s graph position is itself an input to the decision.

2.3 Module and artifact structure

The codebase is deliberately staged so that each module reads the previous module’s artifacts and writes its own. This keeps the pipeline reproducible and makes any single stage independently inspectable.

Module	Responsibility	Key output
01_data_pipeline	Collapse scans to clean legs	legs.parquet
02_graph_construction	Directed graph + stratified weights	edges/nodes/graph
03_bottleneck_audit	Centrality + SLA-breach share	node_metrics.parquet
04_eta_models	Baseline vs graph-enhanced ETA	model_comparison.csv
05_ftl_carting	Route-type decision framework	ftl_carting_decision.csv
06_visualizations	The figure set	outputs/figures/
07_impact_scenario	Top-3-hub upgrade impact	impact_scenario.json
make_memo.js	Operations strategy memo	.docx

Table 1. The pipeline modules, what each owns, and the primary artifact it writes.

3 Technology Stack

The stack favours mature, interpretable tooling over heavyweight dependencies. The one notable substitution is node2vec for GraphSAGE: the brief permits either, and node2vec captures the structural signal we need without pulling in a GPU-grade torch-geometric stack, which keeps the whole pipeline runnable on a laptop in a couple of minutes.

Component	Role
Python 3.12, pandas, NumPy	Data engineering and feature construction.
NetworkX	Directed graph, betweenness, PageRank, clustering.
node2vec + gensim	Structural node embeddings for the ETA model.
XGBoost, scikit-learn	Gradient-boosted ETA / breach models; linear reference.
Matplotlib	The publication figure set.
Streamlit	The interactive operations dashboard.
Node.js (docx)	Programmatic generation of the strategy memo.

Table 2. The stack. NumPy is pinned to 1.26 for gensim compatibility; the entire pipeline is seeded for exact reproducibility.

4 Data Pipeline

4.1 The data model

Each raw row is one scan event recorded as a shipment moves. The crucial structural fact is that, within a single leg, the columns `actual_time`, `osrm_time`, `actual_distance_to_destination` and `osrm_distance` are *cumulative* — they grow scan by scan — and they *reset* at the start of each new leg. The maximum-cumulative row of every (trip, source, dest) group is therefore the leg total. We verified this against the dataset's own factor column, which equals `actual_time ÷ osrm_time` at leg level, and the leg totals reproduced it exactly.

4.2 Collapsing scans to legs

Listing 1. Collapsing scans into leg totals (module 01).

```

1 df = df.sort_values(["trip_uuid", "source_center", "destination_center", "actual_time"])
2 keys = ["trip_uuid", "source_center", "destination_center"]
3 legs = df.groupby(keys, as_index=False).agg(
4     route_type=("route_type", "first"),
5     od_start_time=("od_start_time", "first"),
6     actual_time=("actual_time", "max"),      # cumulative leg total (minutes)
7     osrm_time=("osrm_time", "max"),
8     actual_distance=("actual_distance_to_destination", "max"),
9     osrm_distance=("osrm_distance", "max"))
10 legs["delay_ratio"] = legs["actual_time"] / legs["osrm_time"]
11 legs["sla_breach"] = (legs["delay_ratio"] > 1.20).astype(int)
12 legs["delay_min"] = legs["actual_time"] - legs["osrm_time"]

```

4.3 Cleaning and feature engineering

Cleaning removes self-loops, non-positive times or distances, and physically impossible delay ratios (outside [0.2, 20]) that are clearly data artefacts; this discards only 110 of 26,368 legs. We then engineer the features the rest of the pipeline consumes, summarised below.

Feature	Definition
delay_ratio	$\text{actual_time} \div \text{osrm_time}$ (the core target ratio).
delay_min	$\text{actual_time} - \text{osrm_time}$ (excess minutes).
sla_breach	1 if $\text{delay_ratio} > 1.20$.
tod	night / morning / afternoon / evening from start hour.
dow, is_weekend	day-of-week and weekend flag.
src/dst city, state	parsed from facility names, e.g. "... (Gujarat)".
corridor	"source $\rightarrow$ destination" identifier.
is_ftl	1 for Full-Truck-Load, 0 for Carting.
dwell_min	wall-clock minus transit time (facility handling).
speed_kmph_osrm	free-flow speed implied by OSRM.
log_actual_time	log1p target for the ETA regression.

Table 3. The engineered feature dictionary produced by the data pipeline.

4.4 What the cleaned data says

The headline is unambiguous: the median delay ratio is 2.0 and the mean 2.42, so OSRM under-predicts by roughly half. The breach rate is 94.6% at $> 1.2\times$, 49% at $> 2\times$ and 16% at $> 3\times$. The miscalibration is not uniform, and the variation is itself informative.

Route type	Median ratio	Time of day	Median ratio
FTL	1.92	Evening (best)	1.88
Carting	2.15	Afternoon	2.01
		Night	2.00
		Morning (worst)	2.12

Table 4. Delay ratio by route type and time of day. FTL is more reliable than Carting, and morning dispatches are the worst — both signals the downstream models and framework exploit. Median facility dwell is 49 minutes.

5 Graph Construction

5.1 Nodes and edges

Facilities become nodes, carrying inbound/outbound volume, distinct-corridor counts, and geography. Corridors (source $\rightarrow$ dest) become directed edges. The primary edge weight is the **median delay ratio**; we also persist it stratified by route type and by time of day, exactly as the brief requires, so a single corridor can be read under different operating conditions.

5.2 Edge weights and stratification

Listing 2. Edge weights, stratified by route type and time of day (module 02).

```

1 base = legs.groupby("corridor").agg(
2     median_delay_ratio=("delay_ratio", "median"),
3     n_traversals=("delay_ratio", "size"),
4     total_excess_min=("delay_min", "sum"))
5 rt = legs.groupby(["corridor", "route_type"])["delay_ratio"].median().unstack() # ratio_ftl /
    ratio_carting
6 tod = legs.groupby(["corridor", "tod"])["delay_ratio"].median().unstack() # ratio_morning ...
7
8 G = nx.DiGraph()
9 for _, e in edges.iterrows():
10     G.add_edge(e.source_center, e.destination_center,
11               delay_ratio=e.median_delay_ratio, volume=int(e.n_traversals),

```

```
12 excess_min=e.total_excess_min, cost=max(e.median_osrm, 0.1))
```

5.3 Connectivity

The resulting graph has $|V| = 1,657$ and $|E| = 2,775$, of which 1,934 corridors have at least three traversals and are treated as “stable” for any support-sensitive statistic. The graph has 64 weakly-connected components, but a single giant component covers 81.7% of nodes — consistent with a national network anchored on a few mega-hubs and fed by many small spokes. We use the median, not the mean, throughout edge construction because the delay distribution has a long right tail that would otherwise let a handful of pathological legs dominate a corridor’s weight.

6 Bottleneck and Corridor Audit

6.1 Centrality measures and what they mean

We compute four structural measures and read each operationally. **Betweenness centrality**, weighted by free-flow time, counts how often a hub lies on the network’s least-time routes; a high value marks a structural chokepoint that traffic must pass through. **PageRank**, weighted by volume, shows where throughput concentrates. **In/out degree** captures fan-in and fan-out connectivity. **Clustering** indicates whether a hub is locally bypassable.

Listing 3. Centrality and SLA-breach contribution (module 03).

```
1 btw = nx.betweenness centrality(G, weight="cost", normalized=True)
2 pr = nx.pagerank(G, weight="volume")
3 met["breach_contrib_pct"] = 100 * met["total_excess_min"] / met["total_excess_min"].sum()
4 met["bottleneck_score"] = (0.30*n_btw + 0.20*n_pr + 0.20*n_vol
5                           + 0.15*n_delay + 0.15*n_excess) # normalized blend
```

6.2 The composite bottleneck score

A genuine bottleneck is structurally central *and* high-throughput *and* slow. The composite score blends the normalised measures with weights that favour structural position while still requiring real delay and volume. For ranking hubs in the memo, however, we lead with the single most business-readable quantity: each hub’s *share of total network excess-delay minutes*, with centrality used to explain *why* a hub earns its place.

6.3 Top bottleneck hubs

Hub	State	Delay share	Betw.	In/Out	Delay×
Gurgaon (Bilaspur HB)	Haryana	12.5%	0.191	45 / 49	1.92
Bangalore (Nelamangla)	Karnataka	6.3%	0.089	36 / 35	1.74
Bhiwandi (Mankoli HB)	Maharashtra	5.5%	0.061	29 / 29	2.25
Kolkata (Dankuni HB)	West Bengal	3.7%	0.090	24 / 22	2.59
Hyderabad (Shamshabad)	Telangana	2.8%	0.111	30 / 27	1.90
Top 5 combined		30.8%			

Table 5. Top five bottleneck hubs by share of total network excess delay. These map onto India’s real trunk geography; Gurgaon (Delhi NCR) alone is one-eighth of all network delay.

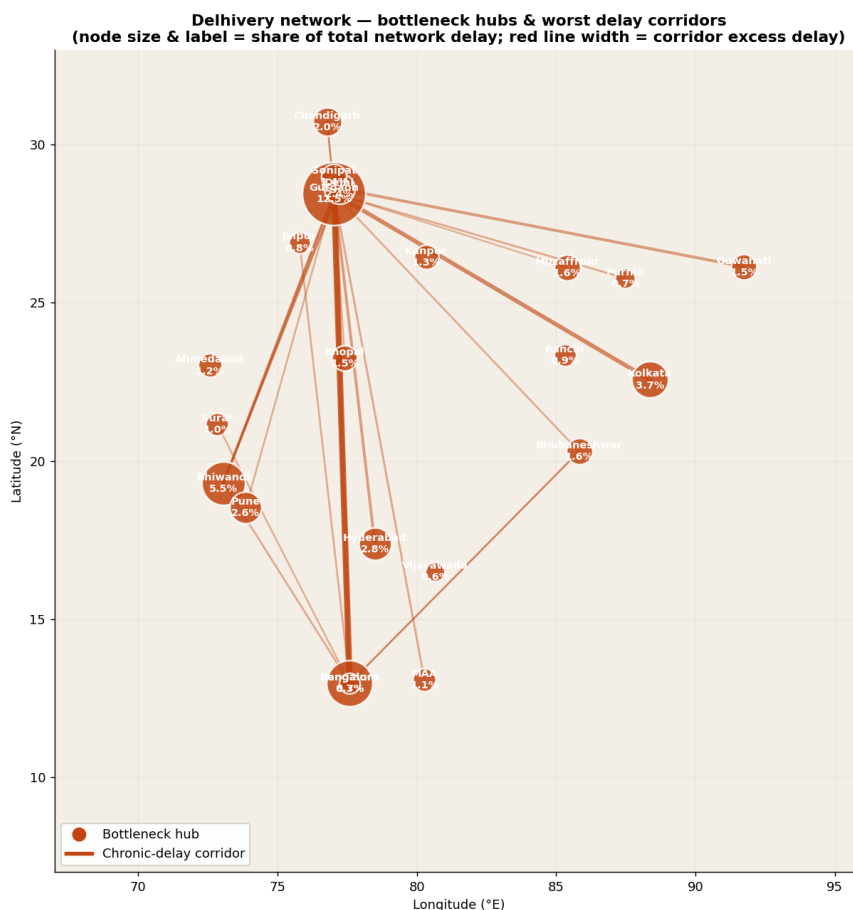


Figure 3. The delay map. Circle size is a hub's share of total network delay; red line width is corridor excess delay. The Gurgaon⇌Bangalore trunk dominates the picture.

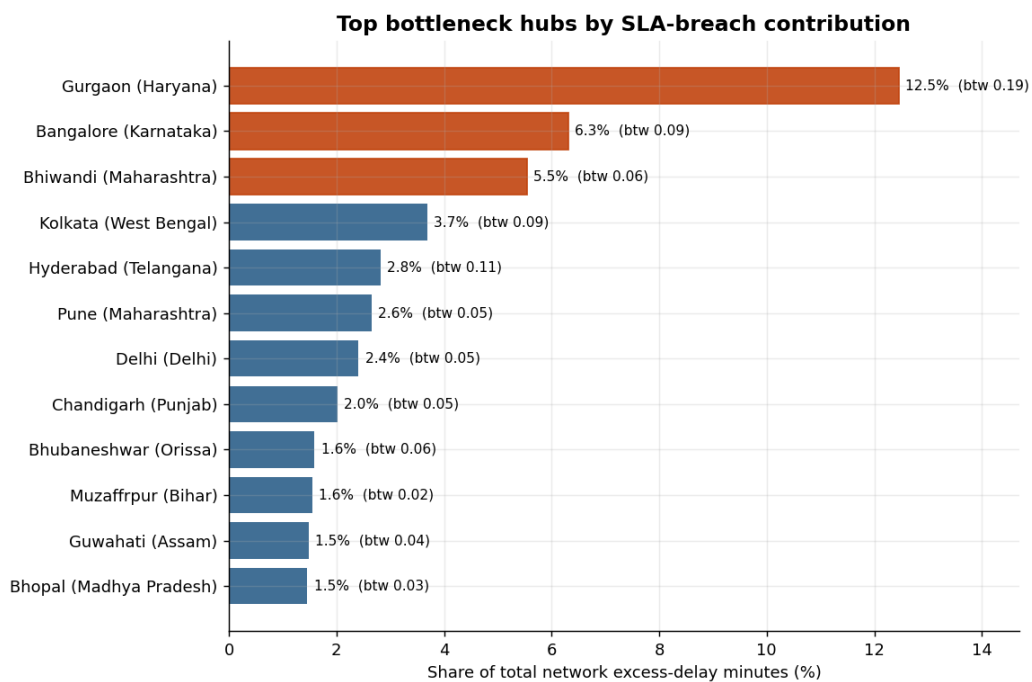


Figure 4. Hub ranking by share of total network delay; the top three are highlighted in orange. The contribution falls off sharply after the leading hubs — the network is genuinely top-heavy.

6.4 Chronically delayed corridors

Ranking stable corridors by total excess minutes surfaces the trunk lanes that bleed the most time. Gurgaon↔Bangalore loses ~145,000 excess minutes across both directions in three weeks; Gurgaon→Kolkata loses 55,636; and Guwahati→Delhi runs at $2.53\times$ OSRM. The full ranking is in Appendix A. These are the first candidates for parallel routing and relay points.

7 Graph-Enhanced ETA Prediction

7.1 Target and features

We predict $\log(1 + \text{actual_time})$, exponentiating back for evaluation; the log target tames the long right tail and stabilises the regression. The **baseline** uses trip features only. The **graph-enhanced** model adds graph-derived features, listed below.

Group	Features
Baseline (trip)	osrm_time, osrm_distance, actual_distance, speed_kmph_osrm, start_hour, dow, is_weekend, is_ftl, tod_code.
Graph priors	corridor median delay ratio, corridor median actual time, corridor support count (train-only).
Graph structure	source/dest betweenness, source/dest PageRank, source out-degree, dest in-degree.
Embeddings	24-dimensional node2vec vectors for both the source and destination facility.

Table 6. Feature groups. The graph priors and embeddings are precisely the network-memory the baseline cannot see.

7.2 Keeping the comparison honest

Leakage control. Every corridor/graph feature is learned on the *train fold only*, then merged onto test with a corridor→state-pair→global cold-start fallback, and node2vec is trained on the *train graph only*. A test leg’s own actual time never enters its features. The “graph advantage” is therefore measured, not leaked.

Listing 4. Leakage-safe graph features and node2vec (module 04).

```

1 G = build_train_graph(train)                                # graph from TRAIN legs only
2 btw = nx.betweenness_centrality(G, weight="cost", normalized=True)
3 emb = Node2Vec(G, dimensions=24, walk_length=15, num_walks=10,
4           workers=1, seed=42, weight_key="volume").fit(window=5).wv
5 corr_prior = train.groupby("corridor").agg(                  # TRAIN-only corridor memory
6     prior_actual=("actual_time", "median"), prior_ratio=("delay_ratio", "median"))

```

7.3 Models and results

Model	MAE (min)	HIT@15%	HIT@25%
OSRM (current)	107.8	4.1%	8.4%
Ridge baseline	198.3	18.8%	32.3%
XGBoost baseline (trip)	34.2	45.2%	65.1%
XGBoost graph-enhanced	27.8	58.4%	74.9%
Graph advantage vs baseline		−18.7% MAE	+13.2 pts

Table 7. Model comparison on the held-out test set ($n = 5,252$). HIT@15% is the share of legs whose predicted ETA lands within 15% of actual — the business metric. The graph model beats both OSRM and the trip-only baseline on every column.

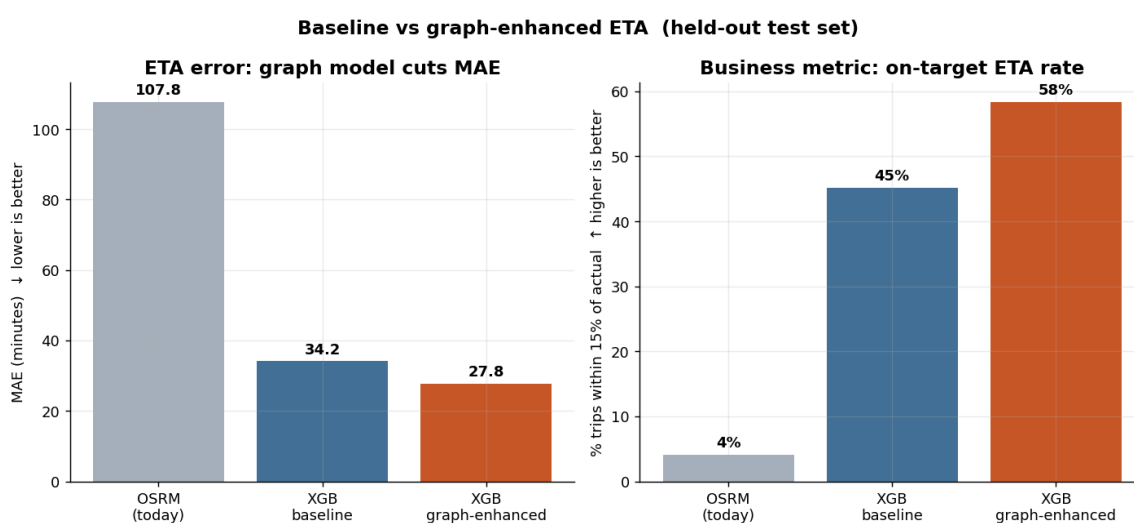


Figure 5. The graph model cuts MAE and roughly triples the on-target-ETA rate versus today's OSRM, and clearly beats a trip-features-only baseline.

7.4 What drives the model

The feature importances are themselves a finding. The single most powerful signal is *corridor history* — the median actual time the corridor has taken before — which a point-to-point engine like OSRM ignores entirely. node2vec embeddings come next, supplying structural generalisation for corridors with thin history. OSRM's own time and distance, the only inputs the current system uses, rank far lower. In other words, the network remembers; independent estimates do not.

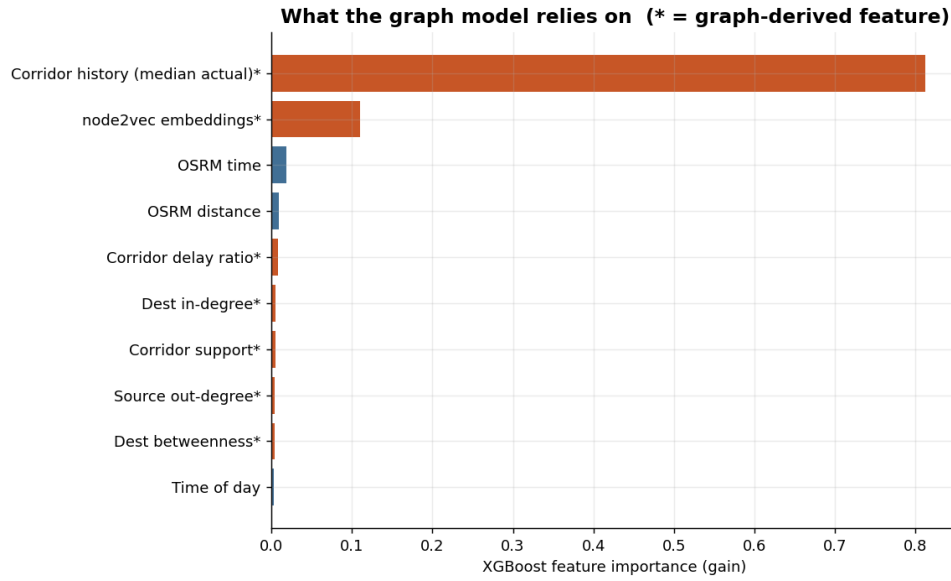


Figure 6. What the graph model leans on. Corridor history dominates, followed by node2vec structural embeddings (* marks graph-derived features).

8 FTL versus Carting Decision Framework

8.1 The confounding problem

Route type is not assigned at random: FTL already runs the long, slow trunks and Carting the short feeders. Predicting absolute time with the route flag naively flipped therefore confounds “where FTL is used” with “what FTL does”. We instead model the *delay ratio* (which normalises out distance) counterfactually, and reconstruct $\text{time} = \text{OSRM} \times \text{ratio}$ for a *fixed* corridor. In the data FTL runs at $1.92\times$ versus Carting’s $2.15\times$, so on the same lane FTL is genuinely more reliable — a clean, defensible signal.

8.2 The cost model

Because the dataset carries no monetary fields, the cost model is explicit and finance-tunable; the framework, not the exact rupees, is the deliverable. FTL is modelled as cheaper per kilometre (a full truck’s efficiency) but with a higher fixed dispatch cost; Carting is cheap to inject into the shared network but pricier per kilometre.

Parameter	Value	Meaning
FTL fixed	Rs 2,500	dedicated-vehicle dispatch cost.
FTL per km	Rs 22	full-truck variable rate.
Carting fixed	Rs 400	cost to add a consignment to the network.
Carting per km	Rs 30	shared/part-load variable rate.
Value of time	Rs 40/hr	working-capital + service value of transit time.
SLA penalty	Rs 1,500	expected cost of a breached leg (penalty + churn).

Table 8. Cost assumptions. The money break-even sits near 260km; time and reliability value then shift the boundary shorter on SLA-critical lanes.

8.3 The generalized cost and decision

For each corridor we score both route types by a generalized cost:

$$\text{GC} = \underbrace{\text{money}(d)}_{\text{fixed+per-km}} + \text{VOT} \cdot t_{\text{hr}} + \text{SLA} \cdot (1 + 3 b_{\text{src}}) \cdot P(\text{breach}),$$

where b_{src} is the source hub's betweenness, so reliability is worth more when the lane originates at a central hub feeding SLA-critical trunks.

Listing 5. Counterfactual scoring of every corridor under both route types (module 05).

```

1 def score(is_ftl):
2     ratio = ratio_model.predict(X.assign(is_ftl=is_ftl))      # delay-ratio counterfactual
3     time = osrm * ratio                                       # reconstruct on fixed lane
4     p = breach_model.predict_proba(...)[0,1]
5     money = ftl_cost(dist) if is_ftl else cart_cost(dist)
6     return money + VOT*time/60 + SLA*(1 + 3*src_btw)*p        # generalized cost
7     recommend = "FTL" if gc_ftl <= gc_cart else "Carting"

```

Distance band	Corridors	% FTL	Avg time saved by FTL
<50 km	1,041	0%	1.7 min
50–150 km	621	0%	2.4 min
150–400 km	188	31%	79 min
400–800 km	36	100%	175 min
>800 km	48	100%	365 min

Table 9. Route-type recommendation by distance band (1,934 stable corridors). The rule that emerges: Carting on short feeders, FTL on long-haul, and a graph-position tiebreaker in the 150–400 km band.

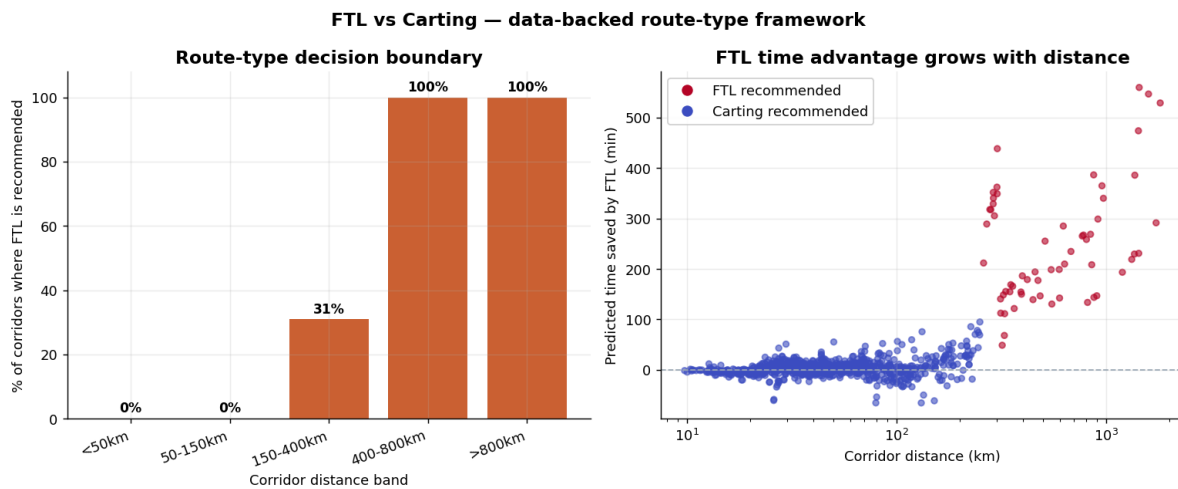


Figure 7. Left: the decision boundary — FTL dominates beyond 400 km, Carting below 150 km. Right: FTL's time advantage grows with distance (red points are where FTL is recommended).

9 A Worked Example: Gurgaon → Bangalore

To make the whole pipeline concrete, follow the single worst corridor in the network end to end. Gurgaon (Bilaspur HB) and Bangalore (Nelamangla H) are 2,065 km apart by OSRM distance.

- Legs.** 66 traversals of this corridor survive cleaning. Their median leg time is 2,866 minutes against an OSRM estimate of 1,548 — a delay ratio of 1.85 and 89,334 total excess minutes, the largest of any single corridor.
- Edge.** The corridor becomes one directed edge weighted by that 1.85 ratio. It is observed entirely as FTL in the data, so its route-stratified weight exists only for FTL.
- Nodes.** Gurgaon carries the network's highest betweenness (0.191), 49 outbound and 45 inbound corridors, and a volume of 1,989 — the structural reason it heads the bottleneck ranking. Bangalore is the southern anchor (betweenness 0.089, volume 1,433).

4. **ETA model.** Because the corridor's history (median actual 2,866) is the model's strongest feature, the graph-enhanced model anchors its prediction on that memory rather than on the misleading 1,548-minute OSM figure — exactly the correction the network view buys.
5. **Route type.** The framework reconstructs both options on this fixed lane: FTL $\approx 3,032$ min versus Carting $\approx 3,384$, so FTL saves ~ 352 minutes; and at 2,065 km FTL is also cheaper (Rs 47,931 versus Rs 62,352). The recommendation is **FTL**, matching how the lane is actually run.

One corridor thus exercises every stage: a leg becomes an edge, the edge's endpoints become ranked bottlenecks, the corridor's history drives a corrected ETA, and its distance and origin-centrality drive a route-type decision.

10 The Interactive Dashboard

The analysis ships as a live Streamlit console so operations staff can explore the network without touching the pipeline. It reads only the generated artifacts, opens on four network KPIs, and is organised into four tabs.

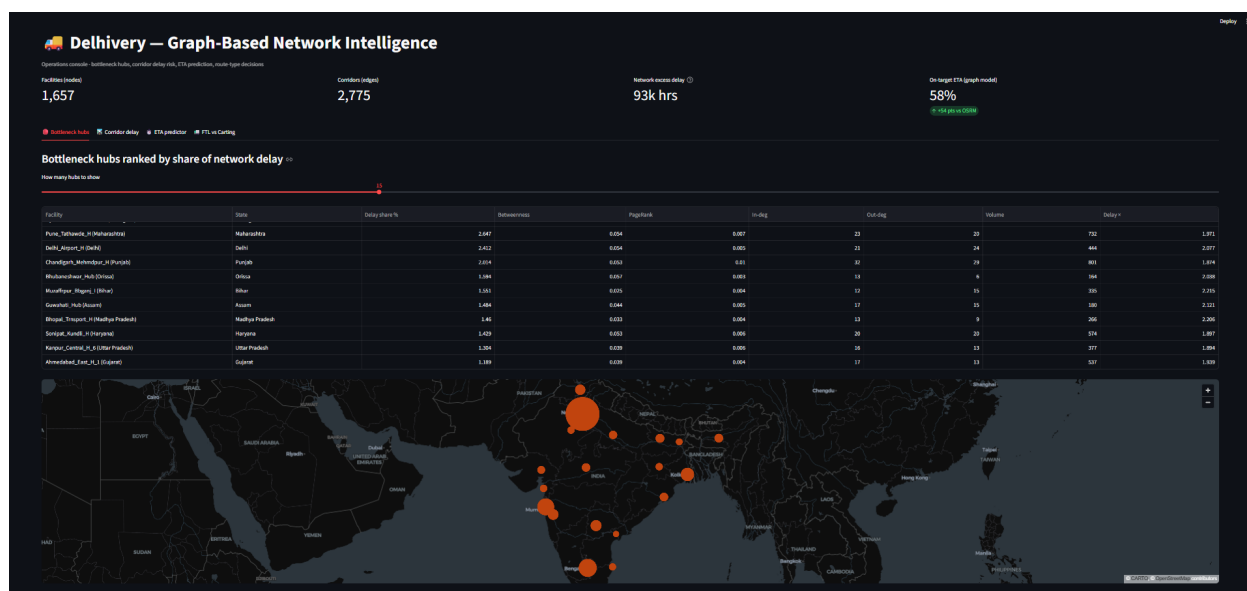


Figure 8. Bottleneck hubs tab. A slider controls how many hubs to rank by share of network delay, with their centrality, degree and volume; below, a live map sizes each hub by its delay contribution. The header KPIs read 1,657 facilities, 2,775 corridors, 93k hours of excess delay, and the 58% on-target ETA.

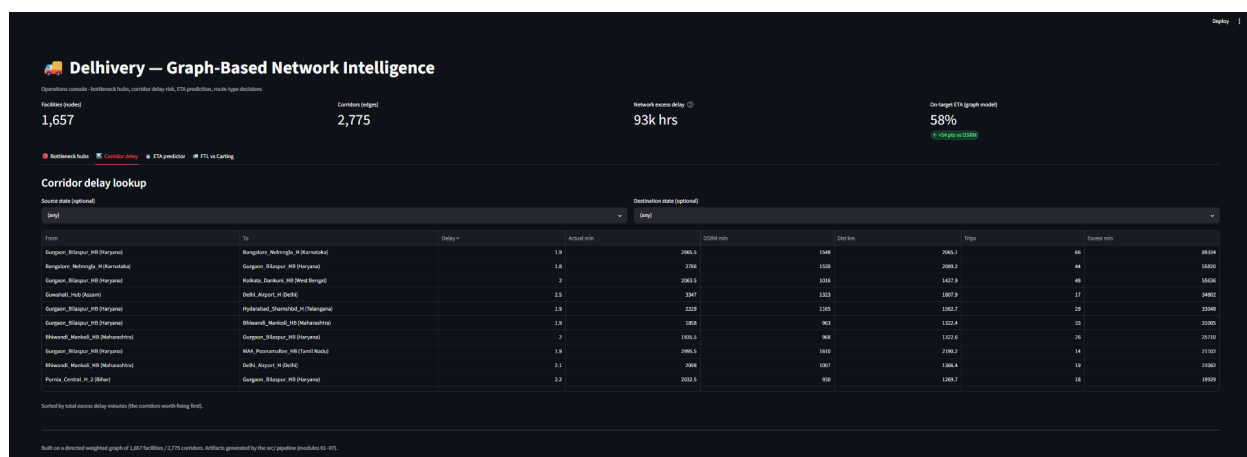


Figure 9. Corridor delay tab. Filter by source/destination state to surface the worst lanes, sorted by total excess delay minutes — the Gurgaon⇌Bangalore trunk tops the list at $\sim 89,000$ excess minutes one way.

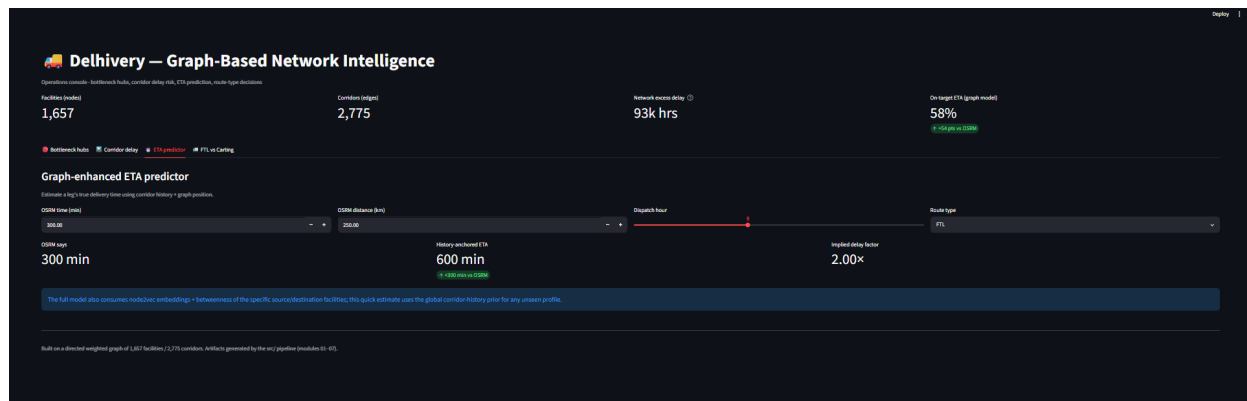


Figure 10. ETA predictor tab. An operator enters an OSM time/distance, dispatch hour and route type; the console returns a history-anchored ETA against the raw OSM figure and the implied delay factor.

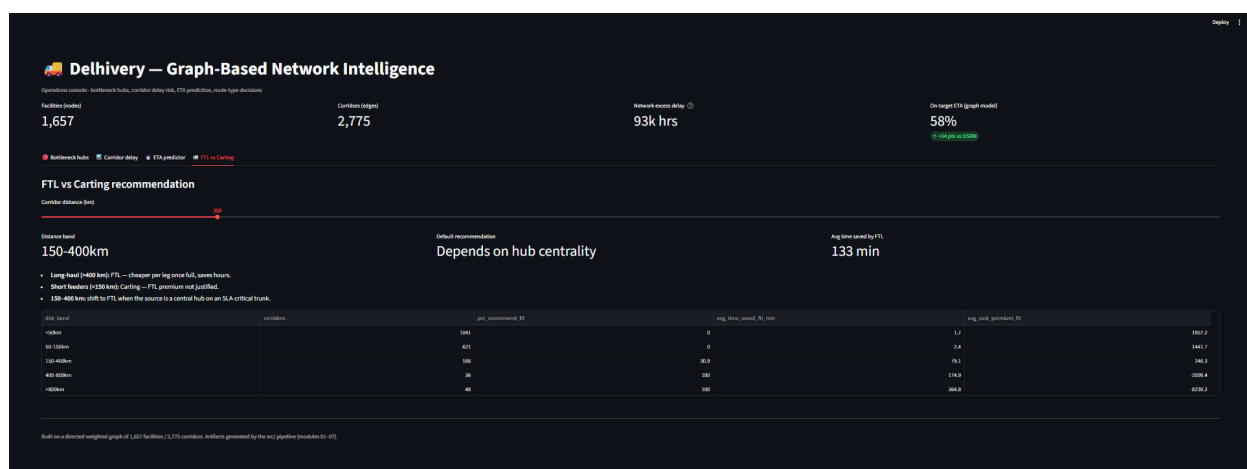


Figure 11. FTL vs Carting tab. A distance slider returns the recommended route type, the average time FTL saves on comparable lanes, and the full band-by-band trade-off table.

Deployment. The app is a single streamlit run app.py away from running locally, and deploys unchanged to Streamlit Community Cloud or any container host.

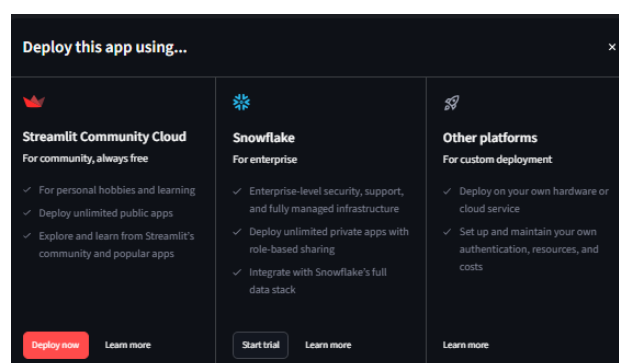


Figure 12. One-click deployment targets exposed by the dashboard.

11 Design Decisions

Several choices shaped the analysis, and each was made deliberately rather than by default.

Median, not mean, for edge weights. The delay distribution has a heavy right tail; the median keeps a corridor's weight representative of typical performance rather than letting a few pathological legs dominate.

Severity $\times$ volume, not the binary breach flag. Because 94.6% of legs breach the literal $1.2\times$ threshold, the flag does not discriminate. Ranking by total excess delay minutes recovers a meaningful priority order.

Betweenness weighted by free-flow time. Using OSRM time as the edge cost makes betweenness reflect the network's natural least-time routes, so high-betweenness hubs are genuine physical chokepoints rather than artefacts of the graph topology.

Log target for ETA. Predicting $\log(1 + \text{time})$ stabilises the regression against the skewed target and improves error on the long lanes that matter most.

node2vec over GraphSAGE. The brief allows either. node2vec captures the structural signal we need without a GPU-grade dependency, keeping the pipeline fast and reproducible.

Delay-ratio counterfactual for route type. Modelling the distance-normalised ratio, rather than absolute time, removes the confound that FTL is already assigned to slow long lanes — the difference between a defensible and a misleading framework.

12 Business Impact and Recommendations

Upgrading the top three hubs (Gurgaon, Bangalore, Bhiwandi) so that 40% of their excess delay is removed — a conservative target that brings them toward network-median performance — yields, on explicit and finance-tunable assumptions:

- **$\sim 28\%$ fewer late deliveries** on the corridors those hubs touch ($\sim 5\%$ across the entire network, from just three sites);
- **$\sim 110,000$ hours/year** of delay recovered — capacity currently lost to congestion and dwell;
- **$\approx \text{Rs } 1.5 \text{ crore/year}$** of revenue-at-risk recovered (SLA penalties and churn avoided).

The recommended sequence for the Head of Network Operations:

1. Deploy the graph-enhanced ETA model for promised-time generation (a software change, no capex).
2. Upgrade Gurgaon first — alone 12.5% of network delay — with parallel sortation and dedicated night-dispatch lanes.
3. Stand up a parallel relay on Gurgaon $\leftrightarrow$ Bangalore, the worst single corridor in the network.
4. Adopt the FTL/Carting rule (FTL on long-haul and central-hub trunks; Carting on short feeders).
5. Review Bhiwandi and Kolkata for facility-side dwell — their delay is high even for their volume, pointing to handling rather than roads.

13 Limitations and Future Work

The system is a working analytical prototype, and several areas are candidates for hardening before a production rollout.

First, the cost and revenue figures rest on stated assumptions rather than monetary data, which the dataset does not contain; the framework is built to be re-run once finance supplies real per-leg costs and SLA penalties. Second, the data spans a single ~ 25 -day window in 2018, so seasonal effects and longer-term drift are out of scope; a production model would retrain on a rolling window. Third, the ETA model is evaluated on a random split; a temporal split would more faithfully simulate forecasting unseen future legs, at the cost of more cold-start corridors. Fourth, node2vec embeddings are transductive — genuinely new facilities require a re-embedding step, which an inductive method such as GraphSAGE would avoid. Finally, the impact scenario models a uniform 40% excess-delay reduction per upgraded hub; a facility-level operational study would replace that with hub-specific targets. None of these affect the core findings; they are the natural next steps for scaling the work.

14 Reproducibility and Conclusion

The pipeline runs in numeric order under a fixed SEED=42 with deterministic node2vec (single worker), so every figure and table in this report regenerates exactly. Each module reads the previous module's artifacts and writes its own, and all business constants live at the top of their files.

Treating Delhivery's network as a graph turns a vague "OSRM is sometimes wrong" into precise, actionable structure: the estimate is $2\times$ low everywhere, the delay concentrates in five hubs, a network-aware model predicts ETAs far better than the routing engine or a trip-only baseline, and route-type choice becomes a rule rather than a habit. The result is a decision system — not just a model — that an operations leader can act on tomorrow.

A Appendix: Worst Chronic Corridors

From	To	Delay $\times$	Actual	OSRM	Excess min
Gurgaon (Bilaspur)	Bangalore (Nelamangla)	1.85	2866	1548	89,334
Bangalore (Nelamangla)	Gurgaon (Bilaspur)	1.82	2786	1530	55,820
Gurgaon (Bilaspur)	Kolkata (Dankuni)	2.03	2064	1016	55,636
Guwahati (Hub)	Delhi (Airport)	2.53	3347	1323	34,802
Gurgaon (Bilaspur)	Hyderabad (Shamshabad)	1.91	2229	1165	33,048
Gurgaon (Bilaspur)	Bhiwandi (Mankoli)	1.93	1858	963	31,005

Table 10. The six corridors that lose the most time versus estimate, over the ~ 25 -day window. Times in minutes.

B Appendix: Environment and Setup

Python 3.12 with pandas, NumPy (1.26, pinned for gensim), scikit-learn, NetworkX, XGBoost, gensim+node2vec and Matplotlib; Node.js with the docx package for the memo; Streamlit for the dashboard. Reproduce with `pip install -r requirements.txt`, then run `python src/01_data_pipeline.py` through `07_impact_scenario.py`, node `src/make_memo.js`, and `streamlit run app.py`.

Om Sharma · Lakshya Chahar · Mayank Bagoria
Graph-Based Network Intelligence for Delhivery